INDABA 2026

3-HOUR WORKSHOP

Automating a Ghana Agrivoltaics Data Pipeline

A 3-Hour Code-Along Workshop · Indaba 2026

Who Is This For

YOU ARE

- → A data practitioner who has written Python or SQL before
- → Someone who has moved data manually -- from CSV to spreadsheet to report
- → Curious about ML pipelines but have not built one end-to-end
- → Working in (or interested in) the African tech ecosystem

YOU'LL LEARN

- → How to design an ELT pipeline from scratch
- → How to write SQL transforms that analysts can trust
- → How to orchestrate tasks with Apache Airflow
- → How to document data provenance for open science

THE GOAL

By the end of this workshop, Ama's pilot data will flow automatically from a field CSV into clean, queryable analytics tables -- every Monday morning, without anyone touching a keyboard.

Take 30 seconds: look at "YOU ARE" -- does this describe you? Good. If you are more senior, I will call out the deeper challenges. If you are newer, every concept builds on the last -- you will not get lost. ---

STORY BEAT

Meet Dr. Ama Asante

She has the data. She has the question. She does not yet have the pipeline.

Dr. Ama Asante is a researcher at the KNUST Responsible AI Lab in Kumasi, Ghana. She runs a 16-week agrivoltaics pilot -- an experiment that places solar panels above crop plots to test whether farms can generate electricity AND grow food on the same land.

THE DATA

- → 3 plots: P1 open sun, P2 raised PV panels, P3 ground-mounted PV
- → 3 crops: tomatoes, chilli pepper, eggplant
- → 112 weekly observations: yield, solar energy, temperature, rainfall, humidity, soil moisture
- → 1 research question: can we feed Ghana and power it from the same field?

HER PROBLEM

Every Monday morning Ama downloads a CSV from the field sensors, pastes it into Excel, writes three VLOOKUP formulas, emails a PDF to her supervisor, and hopes the file did not change columns this week.

Ama is a composite character based on real researchers in the FAIR Forward open data community. Her struggle is real and extremely common. Every concept we build today is a direct answer to a specific pain she has. Keep her in mind. ---

POLL

How does Ama analyse her data right now?

Before we build a solution, let's agree on the problem.

- **A** She runs a Python script she wrote last year
- **B** She uses Excel with manual copy-paste every week
- **C** She has a dashboard that refreshes automatically
- **D** She exports to R and re-runs the analysis each time

ANSWER

B -- and many of you nodded. Manual work is not just slow, it is a reproducibility risk. If Ama gets hit by a bus, the pipeline dies with her.

Use poll feature if available. Otherwise ask for a show of hands. The point is not to shame Excel -- Excel is a great tool. The point is that manual pipelines do not scale, do not reproduce, and do not let anyone else audit what happened. That is what we are fi...

SESSION 1

The Problem

Why Ghana needs both food and energy -- and why that demands a trustworthy pipeline

Ghana's Dual Crisis

Hunger and energy poverty are not separate problems -- they share the same land constraint.

THE FOOD SIDE

- → Ghana imports 40% of its food despite fertile land in the north
- → Smallholder farms lose up to 30% of yield to heat stress and irregular rainfall
- → Climate change is shrinking the reliable growing window each decade

THE ENERGY SIDE

- → 30% of Ghanaians lack reliable electricity access
- → Diesel generators cost rural communities 4-8x the urban tariff
- → Solar capacity targets require land -- land that competes with food production

FOR AMA

- → Her pilot asks: can one parcel of land solve both problems at once?
- → The answer requires data -- clean, comparable, reproducible data

This is not a generic "Africa has problems" framing. The numbers are real. The KNUST pilot exists because these two pressures intersect specifically in Ghana's savannah region. The data Ama collects is the first rigorous attempt to quantify the trade-off. ---

What Is Agrivoltaics?

Same land, two harvests -- one of food, one of electricity.

THE CONCEPT

- → Solar panels mounted above or between crop rows
- → Panels provide partial shade -- reducing heat stress on some crops
- → Rainwater collected by panels can be directed to roots
- → Electricity generated offsets irrigation pump costs

THE THREE PLOTS

- → P1 -- Open sun control: no panels, baseline yield measurement
- → P2 -- Raised PV: panels at 2.5m, crops grow underneath with partial shade
- → P3 -- Ground-mounted PV: panels at 0.5m, crops in narrow inter-row gaps

THE HYPOTHESIS

- → P2 will match or exceed P1 yield while generating solar energy
- → P3 is the stress test: maximum power, minimum crop space
- → Crop response will differ: shade-tolerant vs shade-sensitive species

Draw a quick sketch on the whiteboard if you have one -- or gesture at the slide visual. P1 is our control. P2 is the promising configuration. P3 tests the extreme. We will see in the data whether the hypothesis holds. ---

The KNUST Pilot Dataset

112 rows, 12 columns, 16 weeks of real field observations -- this is what Ama gives us to work with.

THE STRUCTURE

- → observation_id, plot_id, plot_type, crop_type, week_number, observation_date
- → yield_kg_per_m2, solar_energy_kwh
- → temperature_c, rainfall_mm, humidity_pct, soil_moisture_pct

THE LICENCE

- → CC-BY 4.0 -- you can use, share, and adapt with attribution
- → Published via FAIR Forward Open Data Catalog
- → Citation required in any publication or dashboard you ship

THE RESPONSIBLE DATA COMMITMENT

- → We will embed the dataset citation in every output table
- → We will validate every row before it enters our pipeline
- → We will never modify raw data -- only transform copies of it

Demo -- open data/ghana_agrivoltaics.csv in VS Code

Open the CSV now -- it is in the workshop repo under data/. I want everyone to see the raw file before we talk about pipelines. Notice there are no bad rows yet. We will introduce deliberate errors in the exercise to practice defensive loading.

SESSION 2

The Data

What Ama's CSV actually looks like -- and why raw data is never pipeline-ready

Three Plots, Three Crops, Sixteen Weeks

Small dataset, big questions -- every row is a week of someone's fieldwork.

WHAT WE HAVE

- → 3 plots x 3 crops x ~12 weekly observations = 108 core rows
- → Plus 4 supplemental rows for calibration weeks = 112 total
- → Covers March to June -- Ghana's main season dry period

WHAT WE CAN ASK

- → Does yield under P2 panels match open-sun P1 for each crop?
- → Which crop benefits most from partial shade?
- → Does higher solar energy generation correlate with lower yield in P3?
- → Is soil moisture higher under panels -- and does that help?

WHAT WE CANNOT YET ASK

- → Multi-season trends -- 16 weeks is one snapshot
- → Economic viability -- we have kWh but not revenue
- → Generalisation -- one pilot, one site, one season

This "what we cannot ask" section is important. Responsible data science means being clear about the limits of our claims. We will bake those limits into the mart table documentation. ---

What Ama Sees at 6AM

The raw file is not wrong -- it is just not ready. That gap is exactly what a pipeline closes.

THE RAW CSV PROBLEMS

- → Dates in DD/MM/YYYY format -- inconsistent with ISO 8601
- → plot_type is free text: "open sun", "Open Sun", "open-sun" all appear
- → yield_kg_per_m2 occasionally blank when the sensor failed
- → No dataset version column -- which week did Ama download this?

WHAT BREAKS WITHOUT A PIPELINE

- → Two analysts get different results from the same CSV
- → A blank yield row silently becomes zero in an average
- → "Open Sun" vs "open sun" creates a phantom third category
- → No audit trail -- impossible to reproduce Ama's Monday report

FOR AMA

- → She spends 45 minutes every Monday fixing these issues by hand
- → She has never been able to hand the process to a colleague
- → We are going to fix all of this -- in code, once, forever

Every one of these problems is something I have seen in real production systems. The format inconsistencies, the silent nulls, the reproducibility gap. They are not signs of bad researchers -- they are signs of a workflow that has not yet been engineered. ---

POLL

What happens to tomato yield under P2 raised panels vs open sun?

Make a prediction before you see the data -- it sharpens your eye as an analyst.

- **A** Yield drops significantly -- tomatoes need full sun
- **B** Yield stays roughly the same -- partial shade is neutral
- **C** Yield actually increases -- shade reduces heat stress
- **D** It depends on the week -- too variable to generalise

ANSWER

The data shows C for some weeks, D overall -- which tells us something important about how we must aggregate.

Do not reveal the answer immediately -- let the room commit. Then pull up the mart_yield_comparison query result. The point is that intuition fails with agrivoltaics -- the data surprises people consistently. That is why the pipeline matters. ---

DISCUSSION

What questions can this data actually answer?

A pipeline is only worth building if it answers questions someone will act on.

- → Ama's supervisor asks: "Should we scale P2 to 50 farms?" -- can the data answer this?
- → A funder asks: "What is the yield penalty of agrivoltaics?" -- how do we express it?
- → A journalist asks: "Does agrivoltaics work in Ghana?" -- what caveats must we add?
- → A farmer asks: "Which crop should I plant under panels?" -- what does the data say?

Break into pairs for two minutes. I want you to pick one of these questions and write down: (a) what query would you run, and (b) what the answer's limitations are. We will come back to this in Section 5 when we build the mart tables. ---

SESSION 3

Pipeline Architecture

Choosing the right pattern before writing a single line of code

Why Spreadsheets Break at Scale

The problem is not Excel -- the problem is that manual steps do not compose.

WHAT AMA DOES NOW

→ Download CSV, paste into master sheet, run VLOOKUP, email PDF

→ One missed step corrupts the whole chain

→ No version control, no error log, no reproducibility

WHAT BREAKS FIRST

→ A second analyst: they get different numbers -- their Excel rounds differently

→ A new crop type: the VLOOKUP silently returns N/A, Ama does not notice

→ A missed week: the time series has a gap nobody tracks

THE ENGINEERING ANSWER

→ Replace each manual step with a coded, testable, logged function

→ Replace "I emailed it" with "the pipeline ran at 07:00 and wrote to the database"

→ Replace "it should be right" with "I can prove it -- here is the test"

I am not saying Ama is doing it wrong. She is doing the best she can with the tools she has. Our job is to give her better tools -- and to show how those tools connect into a system she can trust and hand off. ---

ELT: Extract, Load, Transform

Load raw data first, transform it later -- raw data is your single source of truth.

WHY ELT NOT ETL

- → ETL transforms before loading -- you lose the original if the transform is wrong
- → ELT loads raw first -- you can re-run any transform without re-extracting
- → Modern databases are cheap enough to store both raw and transformed copies
- → ELT is the industry standard for data warehouses and lakes

OUR THREE LAYERS

- → raw.field_observations -- exact copy of every CSV we ever received
- → staging.observations_clean -- typed, validated, normalised rows
- → mart.* -- business-ready aggregations for analysts and dashboards

KEY DECISIONS

- → We never modify raw -- we only insert new rows
- → Transforms are idempotent -- running twice gives the same result
- → Every mart table carries a dataset_citation column

Draw the three-layer diagram. This architecture decision -- ELT not ETL -- is the most important design choice in this workshop. Every piece of code we write lives in one of these three layers. ---

The Pipeline Blueprint

Six tasks. One DAG. One Monday morning run that Ama never has to touch.

THE SIX TASKS

→ 1. locate_field_csv -- find this week's file on the shared drive

→ 2. validate_observations -- reject rows that fail schema checks

→ 3. upload_to_s3 -- archive the raw CSV to object storage

→ 4. load_to_postgres -- insert validated rows into raw.field_observations

→ 5. run_transformations -- execute SQL transforms for all mart tables

→ 6. publish_reports -- write DATASET_CITATION.txt alongside every output

FOR AMA

→ Each Monday at 07:00 Accra time the DAG runs automatically

→ If any step fails, Ama gets an email -- not a silent wrong answer

→ Every output table carries the CC-BY 4.0 attribution automatically

This is the overview slide. We will implement each task in order over the next 90 minutes. Keep this mental map -- every exercise maps to one of these six tasks. ---

CHECKPOINT

Checkpoint: The Problem and the Plan

Before we write code, let's confirm you have the mental model.

- ✓ Agrivoltaics tests food and energy co-production on the same land
- ✓ Ama's pilot has 3 plots, 3 crops, 16 weeks -- stored in a CSV with data quality issues
- ✓ ELT loads raw first, transforms second -- raw data is never modified
- ✓ Our pipeline has 6 tasks running every Monday at 07:00
- ✓ Every output carries the CC-BY 4.0 dataset citation

Ask for questions before we open a code editor. This is the foundation everything else builds on. If anyone is uncertain about ELT vs ETL, take two minutes now -- it will save confusion in Section 5. ---

SESSION 4

Extract & Load

Getting Ama's CSV into a database she can query

Writing the Extractor

The extractor has one job: find the file and return a clean list of dicts.

```
import csv, pathlib

RAW_DIR = pathlib.Path("data/raw")

def extract(filename: str) -> list[dict]:
    path = RAW_DIR / filename
    if not path.exists():
        raise FileNotFoundError(f"Field CSV not found: {path}")
    with open(path, newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        rows = list(reader)
    print(f"[extract] {len(rows)} rows read from {filename}")
    return rows
```

Demo -- run extract in the notebook

This is 12 lines and it is the entire extractor. The point is that simple functions compose better than complex ones. We will wrap this in a task decorator in Section 6.

Validate Before You Load

A bad row that enters the raw table is much harder to fix than one we reject at the door.

```
REQUIRED = [
    "observation_id", "plot_id",
    "crop_type", "week_number",
    "yield_kg_per_m2"
]

def validate(rows):
    good, bad = [], []
    for row in rows:
        missing = [f for f in REQUIRED
                   if not row.get(f)]
        if missing:
            bad.append({**row,
                        "_errors": missing})
        else:
            good.append(row)
    print(f"[validate] {len(good)} valid"
          f" | {len(bad)} rejected")
    return good, bad
```

Your Turn -- add a check: reject rows where yield < 0

Loading to Postgres

The load step is a controlled INSERT -- no UPDATE, no DELETE, no overwrite.

```
import psycopg2

SQL = """
  INSERT INTO raw.field_observations
    (observation_id, plot_id, plot_type,
     crop_type, week_number, observation_date,
     yield_kg_per_m2, solar_energy_kwh,
     temperature_c, rainfall_mm,
     humidity_pct, soil_moisture_pct,
     loaded_at)
  VALUES (%(observation_id)s, %(plot_id)s,
          %(plot_type)s, %(crop_type)s,
          %(week_number)s, %(observation_date)s,
          %(yield_kg_per_m2)s,
          %(solar_energy_kwh)s,
          %(temperature_c)s, %(rainfall_mm)s,
          %(humidity_pct)s, %(soil_moisture_pct)s,
          NOW())
  ON CONFLICT (observation_id) DO NOTHING
"""

def load(rows, conn_string):
    conn = psycopg2.connect(conn_string)
    cur = conn.cursor()
    cur.executemany(SQL, rows)
    conn.commit(); cur.close(); conn.close()
    print(f"[load] {len(rows)} rows inserted")
```

Demo -- show raw.field_observations in psql after load

EXERCISE

Exercise: Build the Loader

Your first task: get Ama's CSV into Postgres using the pattern you just learned.

DESIGN QUESTIONS

1. **1. What happens if observation_id 112 already exists? Test your ON CONFLICT.**
2. **2. Add a check: reject rows where week_number is outside 1-16**
3. **3. Write the quarantine INSERT -- where do rejected rows go?**
4. **4. How would you prove to Ama that exactly 112 rows loaded correctly?**

12 minutes -- then we regroup

Walk the room. Most people will get the happy path working in 5 minutes. The interesting challenge is question 4 -- a count is not proof, a checksum is. Push the advanced group toward that. ---

CHECKPOINT

Checkpoint: Extract & Load Complete

Three functions down. Raw data is now in Postgres. The hard part begins.

- ✓ extract() reads a CSV and returns a list of dicts -- it raises on missing files
- ✓ validate() separates good rows from bad -- nothing is silently dropped
- ✓ load() inserts with ON CONFLICT DO NOTHING -- safe to re-run any time
- ✓ loaded_at gives us a full audit trail for every row
- ✓ SQL injection is prevented with parameterised queries

Five-minute break after this slide. When we come back we are writing SQL transforms -- the part where the raw data becomes something Ama can actually share with her supervisor. ---

SESSION 5

Transform with SQL

Turning validated observations into answers Ama can act on

The Staging Transform

Staging is where we fix the data quality problems -- once, in SQL, for everyone.

```
CREATE OR REPLACE VIEW staging.observations_clean AS
SELECT
  observation_id,
  plot_id,
  LOWER(TRIM(
    REPLACE(plot_type, '-', ' ')
  ))                    AS plot_type,
  crop_type,
  week_number::INTEGER,
  TO_DATE(observation_date,
          'DD/MM/YYYY')     AS observation_date,
  COALESCE(
    yield_kg_per_m2::NUMERIC, 0
  )                     AS yield_kg_per_m2,
  solar_energy_kwh::NUMERIC,
  temperature_c::NUMERIC,
  rainfall_mm::NUMERIC,
  humidity_pct::NUMERIC,
  soil_moisture_pct::NUMERIC,
  loaded_at
FROM raw.field_observations
WHERE observation_id IS NOT NULL;
```

Demo -- SELECT plot_type, COUNT(*) FROM staging.observations_clean GROUP BY 1

Building the Mart Tables

Four mart tables -- each one answers a specific question Ama's stakeholders will ask.

THE FOUR MARTS

- → mart_yield_by_plot_crop -- average yield per plot per crop per week
- → mart_yield_comparison -- P1 baseline vs P2 vs P3 as percentage difference
- → mart_energy_by_plot -- total kWh generated per plot across all weeks
- → mart_env_conditions -- weekly averages of temperature, rainfall, soil moisture

WHAT AMA'S SUPERVISOR WILL ASK

- → "Which plot produced the most tomatoes?" -- mart_yield_by_plot_crop
- → "Is P2 as productive as open sun?" -- mart_yield_comparison
- → "How much energy did we generate?" -- mart_energy_by_plot
- → "Was this a typical season?" -- mart_env_conditions

THE CITATION COLUMN

- → Every mart table has: dataset_citation TEXT DEFAULT 'CC-BY 4.0 KNUST...'
- → Any dashboard connected to these tables gets attribution automatically
- → This is our CC-BY 4.0 compliance -- baked into the schema

Demo -- open and run src/transforms/mart_yield_comparison.sql

The Key Transform: Yield Comparison

This one query answers Ama's central research question.

```
CREATE TABLE mart.mart_yield_comparison AS
WITH baseline AS (
  SELECT crop_type,
    AVG(yield_kg_per_m2) AS p1_avg
  FROM staging.observations_clean
  WHERE plot_type = 'open sun'
  GROUP BY crop_type
),
all_plots AS (
  SELECT plot_type, crop_type,
    AVG(yield_kg_per_m2) AS avg_yield
  FROM staging.observations_clean
  GROUP BY plot_type, crop_type
)
SELECT
  a.plot_type,
  a.crop_type,
  ROUND(a.avg_yield, 3)     AS avg_yield_kg_m2,
  ROUND(b.p1_avg, 3)        AS baseline_kg_m2,
  ROUND((a.avg_yield - b.p1_avg)
        / b.p1_avg * 100,1) AS yield_pct_vs_baseline,
  'CC-BY 4.0 · KNUST · FAIR Forward'
                            AS dataset_citation
FROM all_plots a
JOIN baseline b USING (crop_type);
```

Demo -- run query, show results sorted by yield_pct_vs_baseline DESC

Run this live. The result table usually surprises the room: chilli pepper yield under P2 often exceeds P1. That finding is what the KNUST pilot was designed to detect.

DISCUSSION

Where would this pipeline fail at 50 farms?

A pipeline that works for one pilot CSV is not the same as a pipeline that works at scale.

- → 50 farms send CSVs with 50 different column name conventions -- what breaks?
- → One farm's sensor goes offline for 3 weeks -- how does the gap appear in the mart?
- → A researcher discovers that week 8 data was corrupted -- how do we reprocess?
- → The KNUST team publishes a new dataset version -- do we overwrite or append?

These are not hypothetical. All four of these happened in the transition from pilot to production in the FAIR Forward catalog. Spend five minutes on scenario 3 -- the reprocessing question is where most pipeline designs fail. The ELT pattern we chose makes repr...

SESSION 6

Orchestration

Making the pipeline run itself -- every Monday, without Ama

The Airflow DAG

Twelve lines of DAG definition replaces Ama's entire Monday morning routine.

```
from airflow.decorators import dag, task
from datetime import datetime

@dag(schedule="0 7 * * 1",
     start_date=datetime(2026, 1, 1),
     tags=["agrivoltaics", "ghana", "elt"])
def ghana_agrivoltaics_pipeline():

    @task()
    def locate_field_csv():
        return find_latest_csv()

    @task()
    def validate_observations(filename):
        return validate(extract(filename))

    @task()
    def upload_to_s3(filename):
        return archive_to_s3(filename)

    @task()
    def load_to_postgres(rows):
        load(rows, conn_string=CONN)

    @task()
    def run_transformations():
        run_all_transforms()

    @task()
    def publish_reports():
        write_citation_file()
```

Demo -- trigger DAG manually in Airflow UI, show Graph View

Airflow Concepts in 90 Seconds

You need five terms to read and write Airflow DAGs confidently.

THE FIVE TERMS

- → DAG -- Directed Acyclic Graph: the pipeline definition
- → Task -- one unit of work: one Python function
- → Operator -- the task template: PythonOperator, BashOperator, SQLExecuteQueryOperator
- → Schedule -- when to run: cron string or @daily or @weekly macros
- → XCom -- how tasks pass data to each other: Airflow's message bus

FOR AMA

- → She sees one green DAG run every Monday in the Airflow UI
- → If it turns red, she gets an email -- not a silent wrong report
- → She can click on any task and see exactly what it did and why it failed

WHAT WE ARE NOT COVERING TODAY

- → Sensors, hooks, providers -- the full Airflow ecosystem
- → KubernetesExecutor for scale -- today we use LocalExecutor
- → Secret management with HashiCorp Vault or AWS Secrets Manager

Airflow has a steep learning curve. Today I am giving you the 20% that covers 80% of what you will use. The Airflow documentation is excellent -- and the Appendix of this deck has links to the sections we skipped. ---

EXERCISE

Exercise: Wire the DAG

You have the functions. Now connect them into a pipeline Airflow can run.

DESIGN QUESTIONS

1. 1. Set the schedule to run every Monday at 07:00 Accra time (UTC+0 in the dry season)
2. 2. If validate_observations fails, should upload_to_s3 still run? Wire accordingly.
3. 3. Add a task that sends Ama an email when run_transformations completes
4. 4. How would you backfill weeks 1-8 if the pipeline was set up in week 9?

15 minutes

Your Turn -- open airflow_dags/ghana_agrivoltaics_pipeline.py

SESSION 7

Trust & Validation

A pipeline Ama's supervisor can publish -- not just run

Data Quality Checks

Validation is not optional -- it is the contract between Ama and her pipeline.

ROW-LEVEL CHECKS

- → observation_id is unique and non-null
- → plot_id in (P1, P2, P3) -- no phantom plots
- → crop_type in (tomato, chilli pepper, eggplant)
- → week_number between 1 and 16
- → yield_kg_per_m2 between 0.0 and 5.0 -- agronomic upper bound

PIPELINE-LEVEL CHECKS

- → Row count after load matches row count in source CSV
- → No observation_id appears more than once in raw.field_observations
- → Each (plot_id, crop_type, week_number) tuple appears exactly once
- → Mart tables are not empty after transform runs

WHAT TO DO WITH FAILURES

- → Quarantine bad rows -- never silently drop them
- → Alert Ama by email with the specific row IDs that failed
- → Let the DAG succeed but mark the run as "partial" in the audit log

Great Expectations and dbt tests are tools that automate much of this. We are writing it manually today so you understand what those tools are doing under the hood. Once you understand it, you will use the tools confidently. ---

What the Pilot Proves -- and What It Does Not

Responsible data science means being as clear about limits as about findings.

WHAT THE DATA SUPPORTS

- → P2 chilli pepper yield matched P1 open-sun in weeks 4-12
- → P2 soil moisture was consistently 8-12% higher than P1
- → P3 yield was 18-24% lower than P1 across all crops
- → P2 generated 2.4 kWh per square metre over 16 weeks

WHAT IT DOES NOT PROVE

- → One season is not a climate signal -- we need 3+ years
- → One site in Kumasi is not Ghana -- soils, rainfall, and latitude vary
- → kWh generated does not equal revenue without a tariff assumption
- → Smallholder adoption depends on upfront cost, not just yield data

HOW WE ENCODE THIS

- → mart table column: data_limitations TEXT -- written by Ama, stored with the data
- → Every report generated by the pipeline includes the caveats section
- → The DATASET_CITATION.txt file links to the full methodology notes

I want to spend two minutes here. This is the responsible AI section. If you build a pipeline that publishes misleading conclusions at scale, the pipeline makes things worse. Ama's caveats must travel with the data -- forever. ---

CHECKPOINT

Checkpoint: The Full Pipeline

Six tasks. Three database layers. One Monday morning run. You built all of it.

- ✓ extract() -- reads the weekly CSV and returns structured rows
- ✓ validate() -- rejects bad rows to quarantine, never silently
- ✓ load() -- idempotent INSERT with ON CONFLICT DO NOTHING
- ✓ staging transform -- normalises formats and fixes data quality issues
- ✓ mart transforms -- four tables that answer Ama's research questions
- ✓ Airflow DAG -- six tasks wired in dependency order, running every Monday

Give the room 30 seconds to look at this list. Everything on it was a problem Ama had this morning. None of it is a problem after today. ---

SESSION 8

Where to Go From Here

From one pilot in Kumasi to a research data platform across West Africa

Where to Go From Here

You now have the foundation. The next steps are about making it production-grade.

THIS WEEK

- → Push the DAG to your team's Airflow instance
- → Connect the mart tables to a BI tool -- Metabase or Evidence.dev
- → Write one Great Expectations suite for the staging layer
- → Share the DATASET_CITATION.txt with Ama's co-authors

THIS MONTH

- → Replace psycopg2 with dbt for the transform layer
- → Add a data lineage graph -- OpenLineage or Marquez
- → Write the methodology section of Ama's next grant proposal using the pipeline's audit log
- → Connect a second site's CSV to the same pipeline

THIS YEAR

- → Multi-site mart -- 10 farms, same schema, one dashboard
- → Automated anomaly detection -- alert when yield drops more than 2 standard deviations
- → Publish the pipeline as an open-source template for agrivoltaics researchers
- → Submit the mart_yield_comparison results to an open-access journal

This is not a wishlist -- every item on this slide is something a team at an African research institution has done in the last 24 months. You have the skills to do it now. The workshop repo has starter code for dbt and Great Expectations in the extras/ folder. ...

WORKSHOP COMPLETE

You Can Now Build Research Data Pipelines

You walked in with a CSV and a research question. You leave with a six-task production pipeline, four mart tables, and an Airflow DAG that runs while you sleep.

→ Agrivoltaics · Open Data · CC-BY 4.0

→ Responsible AI Lab, KNUST · FAIR Forward · Indaba 2026

"The best data pipeline is the one that runs without you -- and leaves a paper trail that proves it did."